

รายงานโครงงาน: โรงแรมอนันต์แบบกระจาย (Distributed Hilbert's Hotel) ด้วยเทคนิค Consistent Hashing

รายวิชา: Object Oriented Data Structure (OODS) — ภาคการศึกษา 1/2569

1. ข้อมูลโครงงานและตารางหน้าที่สมาชิก

1.1 ข้อมูลทั่วไปของโครงงาน

- **ชื่อโครงงาน**: โรงแรมอนันต์แบบกระจาย (Distributed Hilbert's Hotel) ด้วยเทคนิค Consistent Hashing
- **ภาษาและสภาพแวดล้อมการพัฒนา**: Python 3.12.12 (รองรับ Python 3.11+)
- **เครื่องมือและไลบรารีที่ใช้**: Standard Library (`hashlib`, `bisect`, `time`, `tracemalloc`, `math`, `csv`, `statistics`, `dataclasses`) และไลบรารีสนับสนุน (`pytest` 9.2.0 สำหรับทดสอบอัตโนมัติ, `matplotlib` สำหรับสร้างกราฟ, `questionary` สำหรับเมนูโต้ตอบ CLI)

1.2 ตารางหน้าที่และความรับผิดชอบของสมาชิก

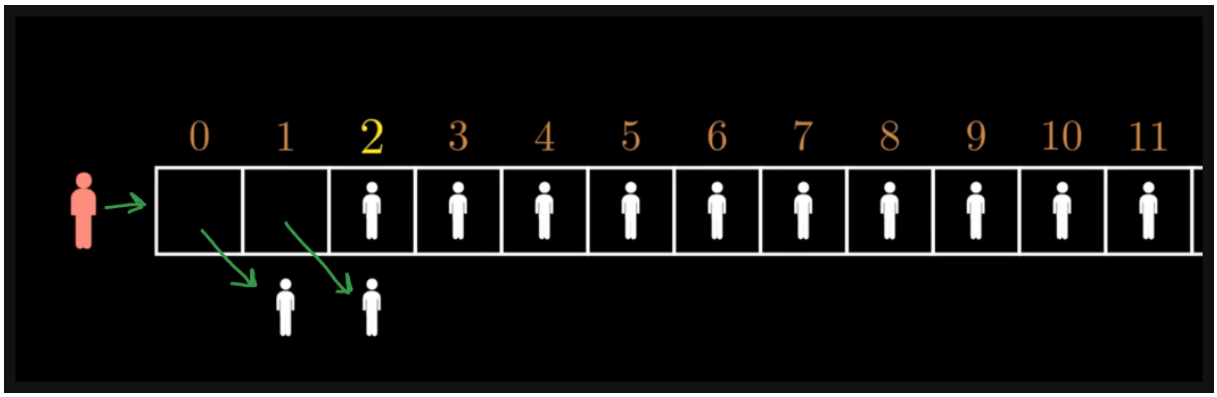
ลำดับ	รหัสนักศึกษา	ชื่อ - นามสกุล	โมดูลที่รับผิดชอบพัฒนา (Modules Developed)	ส่วนที่รับผิดชอบการทดสอบ (Testing Responsibilities)	สัดส่วนการทำงาน (%)
1	68011859	นาย เฮ้า เจิง	`src/hash_ring.py` (`HashRing`), `src/mod_n_system.py` (`ModNHotelSystem`)	`tests/test_task1_ring.py` (7 กรณี: SHA-256 64-bit, Salt, V-Nodes, Mock Ring 100, Tie-Breaker, Mod N Order)	25%
2	68011611	นาย ก้องกิดากร รัตนกูล	`src/guest.py` (`Guest`), `src/indexing.py` (`GuestIndexManager`)	`tests/test_task2_guest.py` (9 กรณี: Cantor Pairing, ความไม่ซ้ำ, Atomic Batch Rollback, ค้นหาสองทาง $\mathcal{O}(1)$)	25%

ลำดับ	รหัสนักศึกษา	ชื่อ - นามสกุล	โมดูลที่รับผิดชอบพัฒนา (Modules Developed)	ส่วนที่รับผิดชอบการทดสอบ (Testing Responsibilities)	สัดส่วนการทำงาน (%)
3	68011738	นาย ปภังกร จึงอนุวัตร	`src/hotel_node.py` (`HotelNode`, `MigrationRecord`), `src/hotel_system.py` (`HotelSystem`)	`tests/test_task3_migration.py` (10 กรณี: กติกาการย้ายเมื่อเพิ่ม/ลบอาคาร, Room Invariance, Load Balance, Edge Cases)	25%
4	68011803	นาย วสุธา เทียนเงิน	`src/export.py` (`CSVExporter`), `src/experiments.py` (`ExperimentRunner`), `src/cli.py` (`HotelCLI`), `src/handlers/`	`tests/test_task4_integration.py` และ `tests/test_smoke.py` (5 กรณี: CSV UTF-8, Migration Log, Guest Generator, Experiment Smoke Test)	25%

2. บทนำและปัญหา (Introduction & Problem Statement)

2.1 ปัญหาคลาสสิกของโรงแรมฮิลเบิร์ต (Classic Hilbert's Hotel Paradox)

ในปัญหาโรงแรมอนันต์คลาสสิกของ Hilbert โรงแรมมีห้องพักจำนวนอนันต์นับได้ $(1,2,3,\ldots)$ และมีแขกเข้าพักเต็มทุกห้อง เมื่อมีแขกใหม่เดินทางมาถึงเพียง 1 คน วิธีแก้ปัญหาเชิงคณิตศาสตร์คือสั่งให้แขกเดิมทุกคนที่พักอยู่ในห้องหมายเลข n ย้ายไปยังห้องหมายเลข $n+1$ พร้อมกัน เพื่อเปิดห้องหมายเลข 1 ให้แก่แขกใหม่



รูปที่ 2.1: การเลื่อนห้องพักแบบลูกโซ่เมื่อรับแขกใหม่ในโรงแรมฮิลเบิร์ตคลาสสิก แขกเดิม 100% ต้องย้ายห้อง ($\Theta(n)$)

อ้างอิง: Veritasium — https://youtu.be/HLTjDXT9SqQ?si=aNtp8j010jgDZwdM&t=45

ข้อจำกัดเชิงระบบ: แม้จะถูกต้องในทางคณิตศาสตร์ แต่ในระบบคอมพิวเตอร์จริง ทุกครั้งที่รับแขกใหม่ 1 คน ระบบต้องย้ายข้อมูลแขกเดิมทั้งหมด K คน คิดเป็นต้นทุนเวลา $\Theta(n)$ หรือ $\mathcal{O}(K)$ ต่อการรับแขกหนึ่งครั้ง ซึ่งก่อให้เกิดภาระมหาศาลและไม่สามารถขยายตัวได้ (Non-scalable)

2.2 ปัญหาการกระจายห้องพักด้วย Naive `hash mod N`

เมื่อขยายโรงแรมเป็นระบบเครือข่ายแบบกระจายศูนย์ (Distributed Hotel) ซึ่งประกอบด้วยอาคาร (Node) จำนวน N หลัง ($N \geq 1$) โดยแต่ละอาคารมีห้องพักไม่จำกัด หากเลือกใช้ฟังก์ชันแฮชหารเอาเศษแบบตรงไปตรงมา:

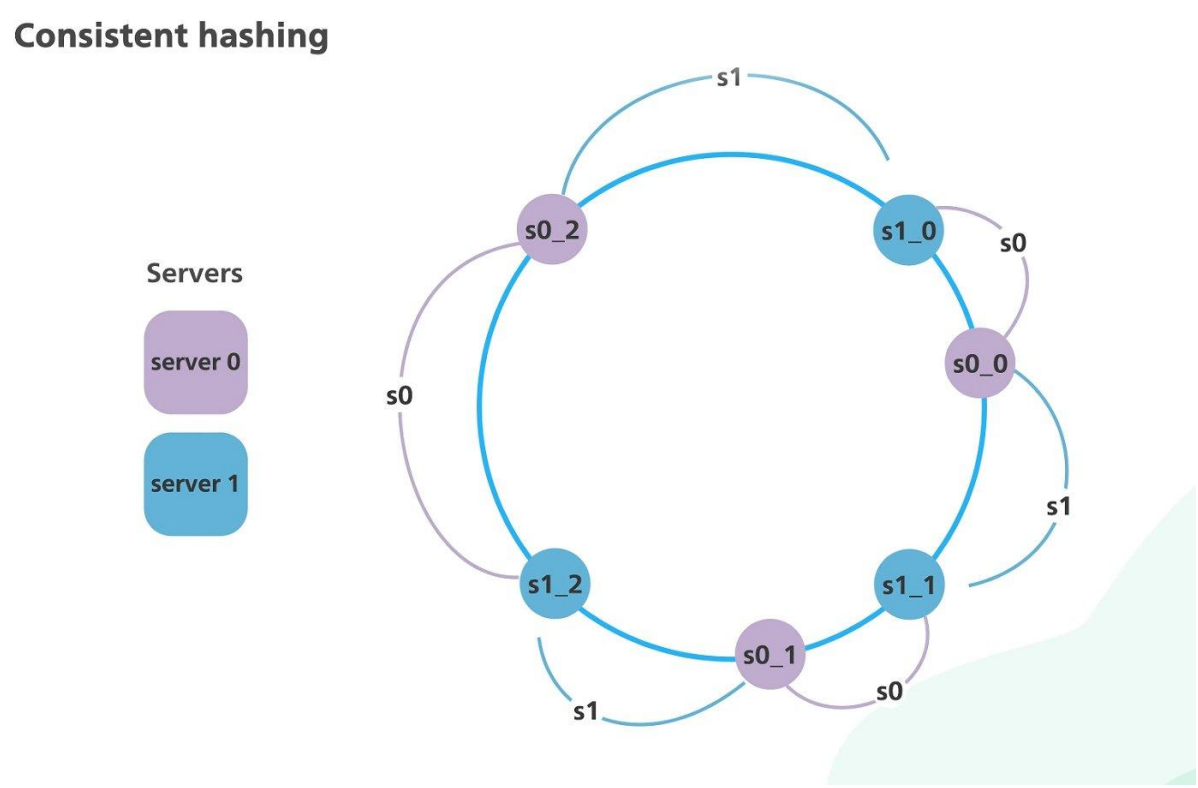
$$\text{node_idx} = H(\text{guest}) \bmod N$$

เมื่อระบบจำเป็นต้องเพิ่มอาคารใหม่เพื่อรองรับภาระงาน ($N \to N+1$) หรือปิดปรับปรุงอาคารเดิม ($N \to N-1$) ตัวหาร N จะเปลี่ยนทันที ส่งผลให้ผลลัพธ์ของสมการมอดูโลของแขกเกือบทั้งหมดคลาดเคลื่อน:

- เมื่อเพิ่มอาคารเป็น $N+1$ สัดส่วนแขกที่ต้องย้ายอาคารจะสูงถึง $\approx \frac{N}{N+1}$ (เช่น ที่ $N = 20$ แขกต้องย้ายถึง $\approx 95\%$ ของแขกทั้งหมด)
- เมื่อลบอาคารเหลือ $N-1$ สัดส่วนแขกที่ต้องย้ายจะสูงถึง $\approx \frac{N-1}{N}$
- พฤติกรรมดังกล่าวทำให้เกิดการย้ายข้อมูลจำนวนมหาศาลพร้อมกัน (Massive Relocation / Cache Stampede) ซึ่งย้อนกลับไปเป็นปัญหาคอขวดแบบ $\mathcal{O}(K)$ เช่นเดียวกับโรงแรมฮิลเบิร์ตคลาสสิก

2.3 ทางออกด้วย Consistent Hashing และ Virtual Nodes

Consistent Hashing แก้ปัญหาการย้ายข้อมูลมหาศาลโดยจำลองพื้นที่ค่าแฮชเป็นวงแหวนปิด (Hash Ring) ขนาดจำนวนเต็มตั้งแต่ 0 ถึง $2^{64}-1$ ($[0, 2^{64}-1]$)
แล้วนำทั้งตำแหน่งของอาคารและรหัสแขกมาวางบนวงแหวนเดียวกัน:



รูปที่ 2.2: แนวคิดวงแหวน Consistent Hashing และการใช้ Virtual Nodes เพื่อลดการย้ายข้อมูลและกระจายภาระงานให้สมดุล

อ้างอิง: ByteByteGo / Hussein Nasser — https://youtu.be/UF9Iqmg94tk?si=F3YcaKtZwmGk0O29&t=402

1. **คุณสมบัติการย้ายข้อมูลน้อยที่สุด** (Minimal Relocation):

   แขกแต่ละคนจะเข้าพักในอาคารเจ้าของจุดแรกที่พบเมื่อเดินตามเข็มนาฬิกา

   - เมื่อเพิ่มอาคารใหม่ 1 หลัง ($N \rightarrow N+1$)

     เฉพาะแขกที่ตกอยู่ในช่วงวงแหวนด้านหน้าจุดของอาคารใหม่เท่านั้นที่ต้องย้าย และต้องย้ายเข้าสู่ **อาคารใหม่เท่านั้น** คิดเป็นจำนวนคาดหวัง $\approx \frac{K}{N+1}$ โดยแขกในอาคารอื่นไม่ถูกรบกวน

   - เมื่อถอนอาคารออก 1 หลัง ($N \rightarrow N-1$)

     เฉพาะแขกที่เคยอยู่ในอาคารที่ถูกลบเท่านั้นที่ต้องย้ายไปยังจุดถัดไปตามเข็มนาฬิกา คิดเป็นจำนวนคาดหวัง $\approx \frac{K}{N}$

2. **โหนดเสมือน** (Virtual Nodes: V): หากวางอาคารละ 1 จุด ($V = 1$)

   ช่วงรับผิดชอบบนวงแหวนอาจมีความยาวไม่เท่ากันมากจนเกิดปัญหาคอขวด (Hotspot) การแทนอาคาร 1 หลังด้วยจุดเสมือนจำนวน V จุดกระจายทั่วงแหวน ช่วยให้ความยาวช่วงรวมของแต่ละอาคารสม่ำเสมอขึ้น ลดค่าสัมประสิทธิ์การแปรผัน ($CV = \sigma/\mu$) ให้ใกล้ความสมดุล

   แลกกับพื้นที่หน่วยความจำสำหรับจัดเก็บจุดบนวงแหวน $M = N \times V$ จุด

3. กติกาการคำนวณและทฤษฎีคณิตศาสตร์ (Mathematical Foundations)

3.1 การคำนวณหมายเลขห้องด้วย Cantor Pairing Function

แขกแต่ละคนในระบบถูกระบุตัวตนอย่างเป็นเอกลักษณ์ด้วยคู่ลำดับ (c, s) โดยที่:

- $c \geq 1$: หมายเลขช่องทางเดินทางมา (Channel ID) เป็นจำนวนเต็มบวก
- $s \geq 1$: ลำดับที่ภายในช่องทางนั้น (Sequence ID) เป็นจำนวนเต็มบวก

ระบบไม่ใช้วิธี "เลขห้องล่าสุดบวกหนึ่ง"

เนื่องจากจะทำให้แขกใหม่ไปกระทบแขกเดิมและทำลายความเป็นอิสระระหว่างอาคาร แต่เลือกใช้ **Cantor Pairing Function** คำนวณหมายเลขห้อง room_no โดยตรงจาก (c, s) ด้วยตัวดำเนินการหารจำนวนเต็ม (`//`) เท่านั้น เพื่อหลีกเลี่ยงความคลาดเคลื่อนของทศนิยม (Floating-point precision loss) เมื่อรหัสมีขนาดใหญ่:

$$a = c - 1, b = s - 1$$

$$\pi(a, b) = \frac{(a + b)(a + b + 1)}{2} + b$$

$$\text{room_no} = \pi(a, b) + 1 = \frac{(a + b)(a + b + 1)}{2} + b + 1$$

ในโค้ดจริง (`src/guest.py` เมธอด `Guest.compute_room_no`) คำนวณด้วยจำนวนเต็มล้วนดังนี้:

```
a = c - 1
b = s - 1
return ((a + b) * (a + b + 1)) // 2 + b + 1
```

การพิสูจน์ความไม่ซ้ำ (Bijective Proof)

กำหนดให้ $a, b \in \mathbb{N}_0$ (จำนวนเต็มไม่ติดลบ) และจัดกลุ่มคู่ลำดับตามแนวทแยงมุมที่ $d = a + b \geq 0$:

1. ในแนวทแยงมุมที่ d มีคู่ลำดับที่เป็นไปได้ทั้งหมด $d + 1$ คู่ ได้แก่ $(d, 0), (d - 1,1), \dots, (0, d)$ ซึ่งมีค่า $b \in \{0,1, \dots, d\}$ ไม่ซ้ำกัน
2. จำนวนคู่ลำดับทั้งหมดในแนวทแยงมุมก่อนหน้า $(0,1, \dots, d - 1)$ คือผลบวกอนุกรมเลขคณิต:

$$T_d = 1 + 2 + \cdots + d = \frac{d(d + 1)}{2} = \frac{(a + b)(a + b + 1)}{2}$$

3. เมื่อบวกตำแหน่งลำดับภายในแนวทแยงมุม $b \in [0, d]$ จะได้ช่วงค่า $\pi(a, b) \in [T_d, T_d + d] = [T_d, T_{d+1} - 1]$ ซึ่งเป็นช่วงจำนวนเต็มที่ไม่ทับซ้อนกันเลยสำหรับแต่ละค่า d และครอบคลุมจำนวนเต็มไม่ติดลบทุกตัวพอดี

4. ดังนั้น $\pi(a, b)$ จึงเป็นการส่งแบบหนึ่งต่อหนึ่งและทั่วถึง (Bijection) จาก $\mathbb{N}_0 \times \mathbb{N}_0 \to \mathbb{N}_0$ และเมื่อบวก 1 จะได้ $\text{room_no} \in \mathbb{N}^+$ ที่ไม่ซ้ำกันทุกคู่ (c, s)

ตารางตรวจสอบตัวอย่างการคำนวณ 5 กรณีมาตรฐาน

รหัสแขก (c, s)	$a = c - 1$	$b = s - 1$	$d = a + b$	$T_d = \frac{d(d+1)}{2}$	$\text{room_no} = T_d + b +$
(1,1)	0	0	0	0	1
(2,1)	1	0	1	1	2
(1,2)	0	1	1	1	3
(3,1)	2	0	2	3	4
(2,2)	1	1	2	3	5

คุณสมบัติคงเดิมข้ามการย้าย (Migration Room Invariance): เนื่องจาก room_no ขึ้นอยู่กับ (c, s) เพียงอย่างเดียว เมื่อมีการเพิ่มหรือลบอาคารแล้วแขกต้องย้ายไปยังอาคารอื่น ระบบจะเปลี่ยนเฉพาะค่า node_id เท่านั้น แต่หมายเลขห้อง room_no ของแขกจะยังคงเดิมเสมอ

3.2 วงแหวนแฮช 64 บิต และการค้นหาตามเข็มนาฬิกา

- **ขอบเขตวงแหวน**: ช่วงจำนวนเต็ม $[0, 2^{64} - 1]$ (0 ถึง $18{,}446{,}744{,}073{,}709{,}551{,}615$)
- **ฟังก์ชันแฮชที่คงที่ข้ามการรัน**: ระบบใช้ `hashlib.sha256` เข้ารหัสข้อความด้วย UTF-8 จากนั้นแปลงผลลัพธ์ฐานสิบหกเป็นจำนวนเต็มและตัดบิตให้เหลือ 64 บิตล่างด้วย Bitmask `&0xFFFFFFFFFFFFFFFF` (โดยห้ามใช้ `hash()` มาตรฐานของ Python เนื่องจากมี Hash Randomization ทำให้ค่าเปลี่ยนทุกครั้งที่เปิดโปรแกรมใหม่):

$$\text{pos}(k) = \text{int}(\text{SHA256}(\text{UTF8}(k + \text{salt})))\&(2^{64} - 1)$$

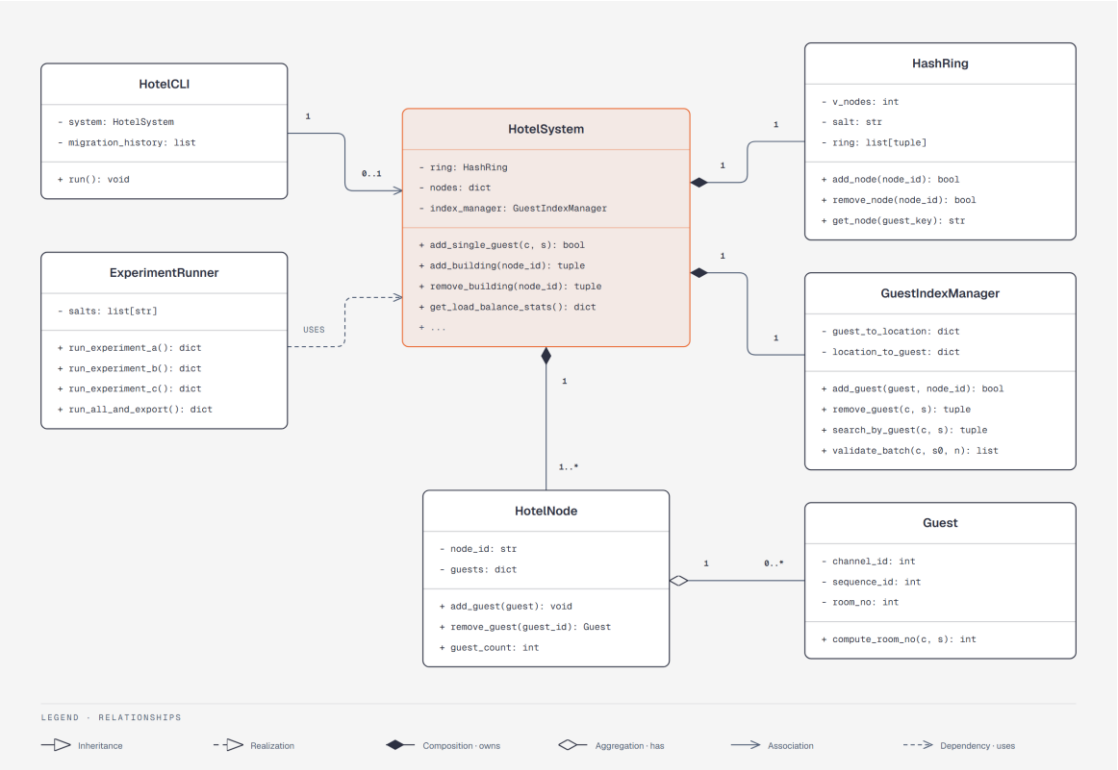
- **รูปแบบคีย์มาตรฐาน (Hash Key Format)**:
  - **คีย์แขก**: `f"guest:{c}:{s}"`
  - **คีย์โหนดเสมือน**: `f"node:{node_id}:{j}"` สำหรับ $j \in \{0,1, \dots, V - 1\}$
  - โดยในแต่ละรอบการทดลองมีการต่อท้ายด้วย `salt` (`""` ในการใช้งานปกติ หรือ `"0"`, `"1"`, `"2"` ในการทดลอง)
- **การค้นหาตามเข็มนาฬิกา (Clockwise Lookup) และ Wrap-around**:

- ระบบเก็บรายการโหนดเสมือนเป็น `list` ที่เรียงลำดับแล้ว และใช้ `bisect_left(self.ring, (guest_pos, "", 0))` เพื่อหาจุดแรกบนวงแหวนที่มี position ≥ guest_pos
- หากดัชนีที่ได้เท่ากับความยาวของวงแหวน (`idx == len(self.ring)`) หมายความว่า guest_pos อยู่เลยจุดสุดท้ายบนวงแหวน ระบบจะ **วนกลับไปที่จุดแรกสุดของวงแหวน** (`idx = 0`)

- **การตัดสินกรณีแฮชชนกัน (Collision Tie-Breaker)**:
  - โครงสร้างข้อมูลใน `self.ring` เก็บเป็นทูเพิล `(position, node_id, j)` และเรียงลำดับตามพจนานุกรมของทูเพิล
  - หากจุดเสมือนหลายจุดแฮชได้ค่า position เท่ากัน ระบบจะเก็บไว้ทุกจุดและเปรียบเทียบด้วย `(position, node_id, j)` เมื่อแขกตกลงที่ตำแหน่งนั้น `bisect_left(self.ring, (guest_pos, "", 0))` จะเลือกจุดแรกที่มี `node_id` น้อยที่สุดตามลำดับอักขระเสมอ

4. การออกแบบเชิงวัตถุและโครงสร้างข้อมูล (OOP Architecture & Data Structures)

ระบบถูกออกแบบตามหลัก **Separation of Concerns (SoC)** แยกตรรกะแกนกลาง (Core Logic) ออกจากส่วนติดต่อผู้ใช้ (CLI Presentation) และการทดลองทางวิทยาศาสตร์อย่างชัดเจน เพื่อให้ทดสอบแยกส่วนได้ง่ายและไม่มีการพึ่งพาแบบวงกลม



รูปที่ 4.1: แผนภาพคลาส (Class Diagram) และความสัมพันธ์ระหว่างโมดูลทั้งระบบ ตั้งแต่ Task 1 ถึง Task 4

4.1 รายละเอียดหน้าที่และโครงสร้างข้อมูลของแต่ละคลาส (Tasks 1–4)

Task	คลาส / โมดูล	หน้าที่ความรับผิดชอบหลัก (Responsibilities)	โครงสร้างข้อมูลที่ใช้ (Data Structures) และเหตุผล
1	`HashRing` (`src/hash_ring.py`)	จัดการวงแหวนขนาด 64 บิต เพิ่ม/ลบโหนดเสมือน V จุดต่ออาคาร แฮชคีย์ด้วย SHA-256 ค้นหาตามเข็มนาฬิกาด้วย Binary Search และรองรับ Mock Ring สำหรับทดสอบ	`ring: list[tuple[int, str, int]]` เรียงลำดับเพื่อใช้ `bisect_left` ค้นหาใน $\mathcal{O}(\log M)$, `node_to_positions: dict[str, list[int]]` สำหรับตรวจสอบอาคารซ้ำใน $\mathcal{O}(1)$ และ `position_to_node: dict[int, str]`
1	`ModNHotelSystem` (`src/mod_n_system.py`)	ระบบเปรียบเทียบพื้นฐาน (Baseline) ที่เลือกอาคารด้วย $H(\text{guest}) \bmod N$ โดยรักษาลำดับอาคารเดิมเมื่อมีการเพิ่มหรือลบอาคาร	`building_list: list[str]` เก็บ `node_id` ตามลำดับที่เพิ่มเข้า เพื่อเข้าถึงด้วยดัชนี $\mathcal{O}(1)$ และคงลำดับสมาชิกที่เหลือเมื่อลบอาคารออก
2	`Guest` (`src/guest.py`)	แทนข้อมูลแขกแต่ละคน ตรวจสอบความถูกต้องของ $(c, s) \geq 1$ คำนวณและแคชหมายเลขห้อง `room_no` ด้วย Cantor Pairing และสร้าง `guest_key`	เก็บ Attributes `channel_id`, `sequence_id`, `room_no` เป็นจำนวนเต็มบวก ลดการคำนวณซ้ำเมื่อต้องอ่าน `room_no` หรือส่งเข้าวงแหวน
2	`GuestIndexManager` (`src/indexing.py`)	จัดการดัชนีค้นหาสองทาง (Bidirectional Index) และตรวจสอบความถูกต้องแบบอะตอมิกเมื่อเพิ่มแขกเป็นกลุ่ม (`validate_batch`)	`guest_to_location: dict[(c, s) -> (node_id, room_no, Guest)]` และ `location_to_guest: dict[(node_id, room_no) -> (c, s)]` ช่วยให้ค้นหาทั้งสองทิศทางได้ใน $\mathcal{O}(1)$ เฉลี่ย
3	`HotelNode` & `MigrationRecord` (`src/hotel_node.py`)	`HotelNode` แทนอาคารแต่ละหลังและเก็บแขกที่พักอยู่ภายใน; `MigrationRecord` เป็น Dataclass บันทึกประวัติการย้ายอาคารของแขกแต่ละคน	`HotelNode.guests: dict[(c, s) -> Guest]` ทำให้ลบอาคารและวนเข้าถึงเฉพาะแขกในอาคารที่ถูกลบได้ทันทีโดยไม่ต้องสแกนแขกทั้งระบบ
3	`HotelSystem` (`src/hotel_system.py`)	ตัวกลางประสานงาน (Orchestrator) เชื่อมโยง `HashRing`, `HotelNode` และ `GuestIndexManager` จัดการเพิ่ม/ลบแขก, เพิ่ม/ลบอาคาร, คำนวณ Migration, วิเคราะห์ Load Balance และเรียงรายการห้อง	`nodes: dict[str, HotelNode]` คู่กับ `ring: HashRing` และ `index_manager: GuestIndexManager` เพื่อให้ทุกโครงสร้างปรับปรุงสอดคล้องกันเสมอ

Task	คลาส / โมดูล	หน้าที่ความรับผิดชอบหลัก (Responsibilities)	โครงสร้างข้อมูลที่ใช้ (Data Structures) และเหตุผล
4	`CSVExporter` (`src/export.py`)	แปลงข้อมูลแขก รายการย้าย และผลสรุปการทดลองเป็นไฟล์ CSV มาตรฐาน UTF-8 (`encoding="utf-8"`)	`list[dict[str, Any]]` ส่งให้ `csv.DictWriter` กรองคอลัมน์ตาม Schema (`channel_id, sequence_id, node_id, room_no` เป็นต้น)
4	`ExperimentRunner` (`src/experiments.py`)	รันการทดลอง A, B, C ครบ 3 รอบด้วย Salt `"0"`, `"1"`, `"2"` แยกวัดเวลาด้วย `time.perf_counter()` และหน่วยความจำด้วย `tracemalloc` คำนวณ Median/Mean/Min/Max และพล็อตกราฟ 300 DPI	`list[float]` เก็บสถิติรายรอบเข้า `statistics.median/mean`, `dict` แบบลำดับชั้นเก็บผลลัพธ์รวมเพื่อส่งออก CSV และพล็อตกราฟด้วย Matplotlib
4	`HotelCLI` & `handlers/` (`src/cli.py`, `src/handlers/`)	ส่วนติดต่อผู้ใช้เมนู 1–10 ด้วย `questionary` แบ่ง Handler เป็น `system.py`, `guest.py`, `building.py` และ `print_box()` โดยไม่แตะคณิตศาสตร์ของวงแหวน	เก็บ Context อ้างอิง `self.system: HotelSystem` และ `self.migration_history: list` ร่วมกันระหว่างทุก Handler

4.2 ตัวอย่างการทำงานรักษาความสอดคล้องของข้อมูล (Data Consistency Walkthrough)

พิจารณาเหตุการณ์เมื่อ **แขกรหัส** $(3,1)$ ซึ่งมีหมายเลขห้องคงที่ $\text{room_no} = 4$ พักอยู่เดิมที่ **`Node_2`** และเกิดการเพิ่มอาคารใหม่ **`Node_3`** จนทำให้แขก $(3,1)$ ต้องย้ายจาก `Node_2` ไปยัง `Node_3` (`HotelSystem.add_building("Node_3")`):

1. **สถานะก่อนการเพิ่มอาคาร (Initial Consistent State)**:
   - `nodes["Node_2"].guests` มีคีย์ `(3, 1) -> Guest(3, 1, room_no=4)` (และ `nodes["Node_3"]` ยังไม่มีอยู่)
   - `index_manager.guest_to_location[(3, 1)] == ("Node_2", 4, Guest(3, 1))`
   - `index_manager.location_to_guest[("Node_2", 4)] == (3, 1)`
   - `search_by_guest(3, 1)` คืนค่า `("Node_2", 4)` และ `search_by_room("Node_2", 4)` คืนค่า `(3, 1)`
2. **ขั้นปรับวงแหวน (`HashRing.add_node("Node_3")`)**:
   - สร้าง `HotelNode("Node_3")` และเพิ่มจุดเสมือน V จุดของ `"Node_3"` ลงใน `self.ring` แล้วเรียงลำดับใหม่ เมื่อคำนวณ `self.ring.get_node("guest:3:1")` จะได้ผลลัพธ์เป็น `"Node_3"`

3. **ขั้นย้ายแขกและอัปเดตดัชนีแบบสอดคล้อง (Atomic State Synchronization ใน `hotel_system.py` บรรทัดที่ 83–99)**:
    - **ย้ายออกจากอาคารเดิม**: `self.nodes["Node_2"].remove_guest((3, 1))` ลบคีย์ `(3, 1)` ออกจาก `Node_2.guests`
    - **เพิ่มเข้าอาคารใหม่**: `self.nodes["Node_3"].add_guest(guest)` บันทึก `Guest(3, 1)` เข้า `Node_3.guests` (โดยใช้ออบเจกต์ `Guest` เดิมที่ `room_no == 4` คงเดิม)
    - **อัปเดตดัชนีจากรหัสแขก**: `guest_to_location[(3, 1)] = ("Node_3", 4, guest)`
    - **ลบและสร้างดัชนีจากห้องพักใหม่**: ลบคีย์เก่า `location_to_guest.pop(("Node_2", 4), None)` แล้วใส่คีย์ใหม่ `location_to_guest[("Node_3", 4)] = (3, 1)`
    - **บันทึกรายการย้าย**: สร้าง `MigrationRecord(channel_id=3, sequence_id=1, old_node="Node_2", new_node="Node_3", room_no=4)`
4. **สถานะหลังการย้าย (Post-Migration Verification)**:
    - `search_by_guest(3, 1)` → `("Node_3", 4)` (เบอร์ห้อง 4 คงเดิม)
    - `search_by_room("Node_3", 4)` → `(3, 1)`
    - `search_by_room("Node_2", 4)` → `None` (ไม่เหลือข้อมูลค้างในดัชนีเก่า)
    - ผลรวมจำนวนแขกใน `HotelNode` ทุกหลังเท่ากับ `index_manager.total_guests()` เสมอ ไม่มีการซ้ำซ้อนหรือสูญหาย

---

5. การวิเคราะห์ความซับซ้อนเชิงเวลาและพื้นที่ (Big-O Analysis)

5.1 นิยามตัวแปรและสมมติฐานในการวิเคราะห์

- N: จำนวนอาคารปัจจุบัน ($N \geq 1$)
- V: จำนวน Virtual Node ต่อหนึ่งอาคาร ($V \geq 1$)
- M: จำนวนจุดเสมือนทั้งหมดบนวงแหวน โดย $M = N \times V$
- K: จำนวนแขกทั้งหมดที่อยู่ในระบบ ณ ขณะนั้น
- ΔK: จำนวนแขกที่เปลี่ยนอาคารจริงจากการเพิ่มหรือลบอาคาร (เป็นปริมาณข้อมูลที่ย้าย ซึ่งแยกจากการวิเคราะห์เวลาประมวลผล โดยในชุดข้อมูลขนาดเล็กหรือกรณีเฉพาะ ΔK อาจเป็น 0 ได้)

- **สมมติฐานขนาดคีย์คงที่ (Constant Key-Size Assumption)**:
  ในการวิเคราะห์กำหนดให้ความยาวของสตริงคีย์ `node_id`, รหัส `(c, s)`, การคำนวณ SHA-256 (64 ไบต์คงที่ต่อบล็อก) และการคำนวณ Cantor Pairing ใช้เวลาและพื้นที่ $\mathcal{O}(1)$ อย่างไรก็ตาม ในภาษา Python จำนวนเต็ม `int` สามารถขยายขนาดได้ไม่จำกัด (Arbitrary-precision) หาก (c, s) เติบโตเป็น B บิตในเชิงทฤษฎี ต้นทุนการคำนวณเลขห้องและการแฮชจะขยายตาม $\mathcal{O}(B)$ ถึง $\mathcal{O}(B^2)$

5.2 การวิเคราะห์ความซับซ้อนตามโค้ดจริง (ครบทั้ง 6 รายการ)

1) การสร้างวงแหวนเริ่มต้น และการจัดแขกเริ่มต้น (แยกสองขั้นตอน)

- **สร้างวงแหวนเริ่มต้น** (`HotelSystem()` + เรียก `add_building` จำนวน N ครั้ง):
  - ใน `HashRing.add_node(node_id)` มีการวนลูปสร้างจุดเสมือน V จุด ($\mathcal{O}(V)$) ต่อด้วย `self.ring.sort()` และ `_sync_position_to_node()` บนรายการขนาด iV (เมื่อเพิ่มอาคารที่ $i \in \{1, \dots, N\}$)
  - **กรณีเลวร้ายที่สุด (Worst-case)**: `sort()` บนขนาด iV มีขอบเขต $\mathcal{O}(iV\log(iV))$ เมื่อเรียก N ครั้งต่อเนื่อง ผลรวมคือ $\sum_{i=1}^{N} \mathcal{O}(iV\log(iV)) = \mathcal{O}(NM\log M)$
  - **พฤติกรรมจริงของ Timsort (Practical / Adaptive)**: เนื่องจากรายการก่อนหน้า $(i-1)V$ จุดเรียงลำดับอยู่แล้ว Timsort ผสานช่วงที่เรียงแล้วกับจุดใหม่ V จุดใน $\mathcal{O}(iV + V\log V)$ ทำให้ต้นทุนรวมของการสร้างทีละอาคารเป็น $\mathcal{O}(NM + M\log V)$ (เทียบกับการสร้างพร้อมกันแล้ว `sort()` ครั้งเดียวที่จะเป็น $\mathcal{O}(M\log M)$)
- **จัดแขกเริ่มต้น K คน** (เรียก `add_single_guest(c, s)` จำนวน K ครั้ง):
  - แต่ละครั้งคำนวณ Cantor $\mathcal{O}(1)$, ตรวจสอบดัชนีซ้ำ $\mathcal{O}(1)$ เฉลี่ย, ค้นหาบนวงแหวนด้วย `bisect_left` $\mathcal{O}(\log M)$, และบันทึกเข้า 3 ดิกชันนารี $\mathcal{O}(1)$ เฉลี่ย
  - **ค่าเฉลี่ย / ค่าคาดหวัง**: $\mathcal{O}(K\log M)$ (กรณีเลวร้ายที่สุดเมื่อเกิด Hash Collision ของ Python `dict` ทุกครั้ง: $\mathcal{O}(K^2 + K\log M)$)

2) การเพิ่มแขก 1 คน, เพิ่มแขกเป็นกลุ่ม และนำแขก 1 คนออก

- **เพิ่มแขก 1 คน (`add_single_guest`)**:
  - **ค่าเฉลี่ย**: $\mathcal{O}(\log M)$ จาก `bisect_left` บน `self.ring` (ส่วน `Guest(c, s)` และอัปเดต 3 `dict` ใช้ $\mathcal{O}(1)$ เฉลี่ย; กรณีเลวร้ายที่สุดของ `dict` คือ $\mathcal{O}(K + \log M)$)
- **เพิ่มแขกเป็นกลุ่มจำนวน B คน (`add_guest_batch`)**:

- ขั้นแรก `validate_batch` ตรวจสอบรหัสซ้ำและสร้าง `Guest` ทั้ง B รายการ ใช้เวลา $\mathcal{O}(B)$ เฉลี่ย (หากพบซ้ำแม้แต่ 1 รายการจะคืนค่า `None` ทันทีแบบ Atomic ไม่แก้สถานะเดิม)
- ขั้นที่สอง วนเพิ่มแขกทั้ง B คนเข้าวงแหวนและดัชนี ใช้เวลา $\mathcal{O}(B\log M)$ เฉลี่ย (กรณีเลวร้ายที่สุดเมื่อล้มเหลวตั้งแต่ต้น: $\mathcal{O}(B)$ หรือถ้า `dict` ชนทุกตัว: $\mathcal{O}(B(K+\log M))$)

- **นำแขก 1 คนออก (`remove_guest`)**:
  - ลบข้อมูลจาก `guest_to_location`, `location_to_guest` และ `HotelNode.guests` โดย **ไม่ต้องเรียกค้นวงแหวน**
  - **ค่าเฉลี่ย**: $\mathcal{O}(1)$ (กรณีเลวร้ายที่สุด `dict` collision: $\mathcal{O}(K)$)

3) การค้นหาผ่านรหัสแขก, ค้นหาผ่านที่อยู่ห้อง และคำนวณอาคารจากวงแหวนโดยไม่ใช้ผลที่บันทึกไว้

- **ค้นหาผ่านรหัสแขก (`search_by_guest`)**: ค้นใน `guest_to_location` ⇒ $\mathcal{O}(1)$ **เฉลี่ย** ($\mathcal{O}(K)$ กรณีเลวร้ายที่สุดของ `dict`)
- **ค้นหาผ่านที่อยู่ห้อง (`search_by_room`)**: ค้นใน `location_to_guest` ⇒ $\mathcal{O}(1)$ **เฉลี่ย** ($\mathcal{O}(K)$ กรณีเลวร้ายที่สุดของ `dict`)
- **คำนวณอาคารจากวงแหวนโดยไม่ใช้ผลที่บันทึกไว้ (`ring.get_node`)**:
  - Consistent Hashing: SHA-256 + `bisect_left` บนรายการขนาด M ⇒ $\mathcal{O}(\log M)$ **ทั้งกรณีเลวร้ายที่สุดและค่าเฉลี่ย**
  - Naive `hash mod N` (`ModNHotelSystem.assign`): SHA-256 + มอดูโล `% len(building_list)` + เข้าถึงอาร์เรย์ ⇒ $\mathcal{O}(1)$ **ทั้งกรณีเลวร้ายที่สุดและค่าเฉลี่ย**

4) การเรียงรายการห้อง รวมต้นทุนการสร้างรายการผลลัพธ์ (`list_occupied_rooms`)

- เรียงชื่ออาคาร `sorted(self.nodes.keys())` ใช้ $\mathcal{O}(N\log N)$
- ภายในแต่ละอาคาร i ที่มีแขก k_i คน ($\sum_{i=1}^{N} k_i = K$) เรียงแขกตาม `room_no` ด้วย Timsort ใช้ $\mathcal{O}(k_i \log k_i)$ และสร้าง `dict` ผลลัพธ์ลงใน `rooms` รวม $\mathcal{O}(K)$
- **กรณีเลวร้ายที่สุด** (แขกทั้งหมดกระจุกในอาคารเดียว): $\mathcal{O}(N\log N + K\log K)$
- **ค่าคาดหวังเมื่อโหลดสมดุล** ($k_i \approx K/N$): $\mathcal{O}(N\log N + K\log(K/N) + K)$

5) การเพิ่มและลบอาคาร (แยก 3 ขั้นตอนตามโค้ดจริง)

- **การเพิ่มอาคาร (`HotelSystem.add_building`,** Method A / แบบ ก):

- **(ก) การปรับวงแหวน (`HashRing.add_node`)**: เพิ่ม V จุดแล้ว `sort()` วงแหวนขนาด $M' = (N+1)V$ พร้อมสร้าง `position_to_node` ใหม่ ⇒ $\mathcal{O}(M\log M)$ **กรณีเลวร้ายที่สุด** ($\mathcal{O}(M + V\log V)$ ตาม Timsort ในทางปฏิบัติ)
- **(ข) การหาแขกที่ได้รับผลกระทบ**: คัดลอก `list(guest_to_location.items())` ($\mathcal{O}(K)$) แล้ว **วนตรวจแขกครบทั้ง K คน** เพื่อเรียก `ring.get_node()` ทุกคน ⇒ $\mathcal{O}(K\log M)$ **ทั้งกรณีเลวร้ายที่สุดและค่าเฉลี่ย** (ถึงแม้จะมีแขกย้ายจริงเพียง ΔK คนก็ตาม)
- **(ค) การปรับข้อมูลและดัชนีของแขกที่ย้าย**: อัปเดต `HotelNode.guests`, `guest_to_location`, `location_to_guest` และต่อท้าย `migrations` เฉพาะ ΔK คนที่ย้ายเข้าอาคารใหม่ ⇒ $\mathcal{O}(\Delta K)$ **เฉลี่ย** (ค่าคาดหวัง $\Delta K \approx \frac{K}{N+1}$, กรณีเลวร้ายที่สุด $\Delta K \leq K$)
- **รวมเวลา `add_building`**: $\mathcal{O}((M+K)\log M)$

- **การลบอาคาร (`HotelSystem.remove_building`)**:
  - **(ก) การปรับวงแหวน (`HashRing.remove_node`)**: กรองจุดของอาคารที่ถูกลบออกด้วย List Comprehension โดย **ไม่ต้อง `sort()` ใหม่** เพราะลำดับจุดที่เหลือยังคงเรียงอยู่แล้ว และสร้าง `position_to_node` ใหม่ ⇒ $\mathcal{O}(M)$ **ทั้งกรณีเลวร้ายที่สุดและค่าเฉลี่ย**
  - **(ข) การหาแขกที่ได้รับผลกระทบ**: เนื่องจาก `HotelNode` เก็บ `guests` แยกตามอาคาร โค้ดจึงดึง `departing_node = self.nodes.pop(node_id)` ($\mathcal{O}(1)$ เฉลี่ย) แล้ววนเฉพาะแขกใน `departing_node.guests` ที่มีจำนวน ΔK คนโดยตรง ไม่ต้องสแกนแขกอีก $K - \Delta K$ คนในอาคารอื่น!
  - **(ค) การคำนวณอาคารใหม่และปรับดัชนีของแขกที่ย้าย**: สำหรับแขก ΔK คนในอาคารที่ถูกลบ เรียก `ring.get_node()` ($\mathcal{O}(\log M)$) และอัปเดต `HotelNode.guests` ของอาคารปลายทางกับดัชนีทั้งสอง ⇒ $\mathcal{O}(\Delta K\log M)$ **เฉลี่ย** (ค่าคาดหวัง $\Delta K \approx \frac{K}{N}$, กรณีเลวร้ายที่สุดหากแขกทุกคนอยู่ตึกเดียว $\Delta K = K$ จะเป็น $\mathcal{O}(K\log M)$)
  - **รวมเวลา `remove_building`**: $\mathcal{O}(M + \Delta K\log M)$ **เฉลี่ย** (ซึ่งอธิบายได้ว่าเหตุใดในผลการทดลอง B เวลาลบอาคารของ Consistent Hashing จึงเร็วกว่าเวลาเพิ่มอาคารหลายเท่า)

6) ความซับซ้อนเชิงพื้นที่ (Space Complexity)

- **วงแหวน (`HashRing`)**: `ring` ขนาด M ทูเพิล, `position_to_node` ขนาดไม่เกิน M รายการ และ `node_to_positions` รวม M ค่า ⇒ $\mathcal{O}(M) = \mathcal{O}(NV)$

- **ข้อมูลแขกและอาคาร (`HotelSystem.nodes` & `HotelNode`)**: N อาคารและ K ออบเจกต์ `Guest` ⇒ $\mathcal{O}(N + K)$
- **ดัชนีเพิ่มเติม (`GuestIndexManager`)**: `guest_to_location` (K รายการ) + `location_to_guest` (K รายการ) ⇒ $\mathcal{O}(K)$
- **พื้นที่หลักรวมของระบบ (Core Persistent Space)**: $\mathcal{O}(NV + K)$
- **พื้นที่ชั่วคราวระหว่างทำงาน (Temporary Space)**:
  - `add_building`: สำเนา `list(guest_to_location.items())` ใช้ $\mathcal{O}(K)$ และรายการ `migrations` ใช้ $\mathcal{O}(\Delta K)$
  - `remove_building`: รายการใหม่ตอนกรอง `ring` ใช้ $\mathcal{O}(M)$, สำเนา `departing_node.guests` และ `migrations` ใช้ $\mathcal{O}(\Delta K)$
  - `add_guest_batch(c, s_start, B)`: รายการ `new_guests` ใน `validate_batch` ใช้ $\mathcal{O}(B)$
  - `list_occupied_rooms`: รายการผลลัพธ์ `rooms` ใช้ $\mathcal{O}(K)$ และรายการชั่วคราวเรียงห้องต่ออาคารใช้ $\mathcal{O}(\max k_i) \leq \mathcal{O}(K)$

5.3 ตารางเปรียบเทียบ **`hash mod N`** กับ Consistent Hashing (ตามข้อกำหนด 8.2)

ตารางต่อไปนี้แยก **"ค่าจากการวิเคราะห์เชิงทฤษฎีจากโค้ดจริง"** ออกจาก **"ค่าที่วัดได้จริงจากไฟล์ CSV"** โดยรายงานค่ามัธยฐาน (Median) พร้อมช่วงต่ำสุด–สูงสุด (Min–Max) ข้าม 3 รอบ (`salt = "0", "1", "2"`)

ประเด็นเปรียบเทียบ	ค่าจากการวิเคราะห์ (อ่านจากโค้ดจริง)	ค่าที่วัดได้จริง (จาก Experiment A, B, C)
การคำนวณอาคาร (Lookup)	Mod N: $\mathcal{O}(1)$ (SHA-256 + `% N`) CH (Stateless Ring): $\mathcal{O}(\log M)$ (`bisect_left` บน $M = NV$ จุด) CH (Cached Index): $\mathcal{O}(1)$ เฉลี่ย (`guest_to_location`)	Exp A ($N = 10, V = 32$): `search_per_query_time` ของ `system.search_by_guest()` (Cached Index) ได้มัธยฐาน $0.099\ \mu s$ ($K = 1{,}000$), $0.108\ \mu s$ ($K = 10{,}000$), $0.167\ \mu s$ ($K = 100{,}000$) *(หมายเหตุ: โค้ด `experiments.py` วัด `search_by_guest()` ผ่านดัชนี $\mathcal{O}(1)$ ส่วนต้นทุน `ring.get_node()` แบบ Stateless $\mathcal{O}(\log M)$ สะท้อนอยู่ใน `assignment_time` และ `ch_add_time` เฉลี่ย $\approx 1.75\ \mu s$ ต่อคน)*

ประเด็นเปรียบเทียบ	ค่าจากการวิเคราะห์ (อ่านจากโค้ดจริง)	ค่าที่วัดได้จริง (จาก Experiment A, B, C)
การเพิ่ม / ลบอาคาร	Mod N: ปรับ `building_list` ใช้ $\mathcal{O}(1)$ ตอนเพิ่ม / $\mathcal{O}(N)$ ตอนลบ; แต่หากต้องคำนวณย้ายแขกทั้ง K คนใช้ $\mathcal{O}(K)$ CH (**`add_building`**): ปรับวงแหวน $\mathcal{O}(M\log M)$ + สแกน K คน $\mathcal{O}(K\log M)$ + อัปเดตดัชนี $\mathcal{O}(\Delta K)$ CH (**`remove_building`**): กรองวงแหวน $\mathcal{O}(M)$ + คำนวณเฉพาะ ΔK คนในอาคารนั้น $\mathcal{O}(\Delta K\log M)$	Exp B ($K = 10{,}000, V = 32$): • $N = 5$: CH เพิ่ม 18.51 ms, ลบ 4.92 ms | Mod N เพิ่ม 10.84 ms, ลบ 10.70 ms • $N = 10$: CH เพิ่ม 17.46 ms, ลบ 2.96 ms | Mod N เพิ่ม 10.62 ms, ลบ 10.62 ms • $N = 20$: CH เพิ่ม 17.62 ms, ลบ 1.64 ms | Mod N เพิ่ม 10.58 ms, ลบ 10.20 ms *(ลบอาคารใน CH เร็วกว่าเพิ่มอาคาร 3.8–10.7 เท่า เพราะวนเฉพาะ ΔK ใน `departing_node`)*
อัตราการย้าย (Migration Rate)	Mod N: ค่าคาดหวัง $\frac{N}{N+1}$ (เพิ่ม), $\frac{N-1}{N}$ (ลบ) CH: ค่าคาดหวัง $\frac{1}{N+1}$ (เพิ่ม), $\frac{1}{N}$ (ลบ)	Exp B ($K = 10{,}000, V = 32$) [Median (Min–Max)]: • $N = 5$: CH เพิ่ม 0.1821 (0.1456–0.1960), ลบ 0.1901 (0.1803–0.2273) | Mod N เพิ่ม 0.8329 (0.8314–0.8457), ลบ 0.7936 (0.7892–0.7977) • $N = 10$: CH เพิ่ม 0.0872 (0.0719–0.1146), ลบ 0.0973 (0.0813–0.1069) | Mod N เพิ่ม 0.9075 (0.9062–0.9076), ลบ 0.8979 (0.8957–0.9033) • $N = 20$: CH เพิ่ม 0.0549 (0.0405–0.0576), ลบ 0.0576 (0.0303–0.0684) | Mod N เพิ่ม 0.9518 (0.9487–0.9519), ลบ 0.9489 (0.9488–0.9510)
พื้นที่จัดเส้นทาง (Routing Space)	Mod N: $\mathcal{O}(N)$ (เก็บ `building_list` ขนาด N) CH: $\mathcal{O}(NV)$ (เก็บ `ring` ขนาด $M = NV$ และแมปช่วยอีก 2 ชุด)	ในเชิงโครงสร้าง CH เพิ่มพารามิเตอร์ V เท่าของ Mod N (ที่ $N = 10, V = 32$ มี $M = 320$ จุดเทียบกับ 10 ชื่ออาคาร; ส่วนหน่วยความจำระบบรวมที่ `tracemalloc` วัดใน Exp C อยู่ช่วง 4.06–4.08 MiB เนื่องจากข้อมูลแขก $K = 10{,}000$ เป็นสัดส่วนใหญ่กว่าพื้นที่วงแหวน)

ประเด็นเปรียบเทียบ	ค่าจากการวิเคราะห์ (อ่านจากโค้ดจริง)	ค่าที่วัดได้จริง (จาก Experiment A, B, C)
พื้นที่ข้อมูลรวม (Total Space)	Mod N (รวมระบบเก็บแขก): $\mathcal{O}(N+K)$ CH (**`HotelSystem`**): $\mathcal{O}(NV+K)$	Exp A ($N=10, V=32$) [Current / Peak Median]: • $K=1{,}000$: 0.3003 / 0.3005 MiB • $K=10{,}000$: 4.0850 / 4.0852 MiB • $K=100{,}000$: 47.0389 / 47.0391 MiB *(วัดด้วย `tracemalloc` เฉพาะหน่วยความจำที่ Python ติดตาม)*
ความสมดุลของโหลด (Load Balance CV)	Mod N: สม่ำเสมอตามโครงสร้าง (Uniform Buckets) คาดหวัง $CV \approx \sqrt{N/K}$ (≈ 0.0316 ที่ $N=10, K=10{,}000$) CH: ขึ้นกับจำนวนจุดเสมือน V ค่า CV ลดลงประมาณ $\propto 1/\sqrt{V}$	Exp C ($N=10, K=10{,}000$) [Median (Min–Max)]: • $V=1$: $CV=0.8330$ (0.8325–0.9488) • $V=8$: $CV=0.4120$ (0.1696–0.5036) • $V=32$: $CV=0.1371$ (0.0772–0.1549)

6. การจัดการกรณีขอบและการทดสอบความถูกต้อง (Edge Cases & Correctness Verification)

6.1 ตารางกลยุทธ์จัดการกรณีขอบ 8 กรณี (Polite Rejection Strategy)

ระบบใช้หลักการ **Polite Rejection** โดยตรวจสอบเงื่อนไขนำเข้าทุกครั้ง แจ้งเตือนเหตุผลอย่างชัดเจน คงสภาพข้อมูลเดิม 100% และไม่เกิด Uncaught Exception จนโปรแกรมหยุดทำงาน:

กรณีที่	เหตุการณ์ที่เกิดขึ้น (Scenario)	พฤติกรรมของโปรแกรมที่เขียนจริง	ผลต่อสถานะข้อมูล
1	เพิ่มรหัสอาคารที่มีอยู่แล้ว	`HashRing.add_node` และ `HotelSystem.add_building` คืนค่า `False` พร้อมรายการย้ายว่าง `[]` ไม่ใส่ V-Node ซ้ำ	ข้อมูลเดิมคงที่ 100%
2	ลบรหัสอาคารที่ไม่มีอยู่ในระบบ	`HotelSystem.remove_building` คืนค่า `(False, [], 0.0)` แจ้งผู้ใช้ว่าไม่พบอาคาร รายงานคนย้าย 0 คน	ไม่มีการเปลี่ยนแปลง
3	พยายามลบอาคารสุดท้าย ($N=1$)	ตรวจสอบ `len(self.nodes) <= 1` แล้วปฏิเสธทันที คืนค่า `(False, [], 0.0)` รักษากฎ $N \geq 1$	อาคารสุดท้ายยังคงอยู่

กรณีที่	เหตุการณ์ที่เกิดขึ้น (Scenario)	พฤติกรรมของโปรแกรมที่เขียนจริง	ผลต่อสถานะข้อมูล
4	เพิ่มรหัสแขก (c, s) ที่ซ้ำ	ตรวจพบใน `guest_to_location` แล้วคืนค่า `False` ไม่เขียนทับข้อมูลและเบอร์ห้องเดิม	แขกเดิมคงอยู่ห้องและตึกเดิม
5	เพิ่มแขกแบบกลุ่มที่มีบางคนซ้ำ	`validate_batch` ตรวจรหัสซ้ำทั้งหมดก่อน หากพบซ้ำแม้เพียง 1 คนจะคืน `None` ปฏิเสธทั้งกลุ่มแบบ Atomic	สถานะก่อนหน้าไม่เปลี่ยนเลย
6	นำแขกที่ไม่มีอยู่ออก	`GuestIndexManager.remove_guest` คืนค่า `None` แจ้งผู้ใช้ว่าไม่พบ โดยไม่ลดตัวนับแขก	สถิติและจำนวนแขกคงเดิม
7	ค้นหาแขกหรือห้องที่ไม่พบ	`search_by_guest` และ `search_by_room` คืนค่า `None` แล้ว CLI แสดงข้อความ "Not Found" อย่างชัดเจน	โปรแกรมทำงานต่อเนื่องได้
8	ระบบไม่มีแขกเลย ($K = 0$)	`get_load_balance_stats` คืนค่า `max=0, min=0, mean=0.0, std=0.0, cv="N/A"` และอัตราการย้ายคืนค่า `0.0` (ไม่หารด้วยศูนย์)	ไม่เกิด `ZeroDivisionError`

6.2 การทวนสอบด้วยวงแหวนจำลองขนาด 100 (Section 9.1 Toy Ring Verification)

ใช้ `HashRing.set_mock_ring` ทดสอบวงแหวนจำลองขนาด 100 ($V = 1$) โดยมีอาคารเริ่มต้นที่ $A = 20, B = 50, C = 80$ และแขก 7 คนตามตำแหน่งแฮช (ทดสอบจริงใน `test_t1_4_mock_ring_100_lookup` และ `test_t1_5_mock_ring_migration_dynamics`):

ตำแหน่งแฮชของแขก	สถานะเริ่มต้น ($A = 20, B = 50, C = 80$)	เพิ่มอาคาร D ที่ 35	ลบอาคาร B จากสถานะเริ่มต้น
10	A	A	A
20 (ตรงจุด A)	A	A	A
25	B	D (ย้ายเข้า D)	C (ย้ายจาก $B \rightarrow C$)
35 (ตรงจุด D)	B	D (ย้ายเข้า D)	C (ย้ายจาก $B \rightarrow C$)
50 (ตรงจุด B)	B	B	C (ย้ายจาก $B \rightarrow C$)
70	C	C	C
90 (วนกลับต้นวงแหวน)	A	A	A

ตำแหน่งแฮชของแขก	สถานะเริ่มต้น ($A = 20, B = 50, C = 80$)	เพิ่มอาคาร D ที่ 35	ลบอาคาร B จากสถานะเริ่มต้น
สรุปผลการย้าย	—	**ย้าย 2 คน** (เข้าสู่ D เท่านั้น)	**ย้าย 3 คน** (เฉพาะแขกเดิมของ B เข้า C)

6.3 ผลการทดสอบอัตโนมัติด้วย Pytest (Section 9.2 Automated Test Report)

ระบบทดสอบด้วยชุดทดสอบอัตโนมัติทั้งหมด 31 **กรณี** ครอบคลุมข้อกำหนดขั้นต่ำทั้ง 6 หมวดในหัวข้อ 9.2 ของโจทย์โครงงาน ผลการรันด้วยคำสั่ง `uv run pytest` บน Python 3.12.12 (pytest 9.2.0):

- **สรุปผลรวม**: **`31 passed in 0.50s`** (`0 failed`, `0 skipped`)

ไฟล์ทดสอบ	จำนวนกรณี	ขอบเขตการตรวจสอบหลัก	สถานะ
`tests/test_smoke.py`	1	ตรวจสอบความพร้อมของสภาพแวดล้อมการทดสอบ	ผ่าน (1/1)
`tests/test_task1_ring.py`	7	SHA-256 64-bit คงที่ข้ามอินสแตนซ์, Salt, V-Node $V = 32$, Mock Ring 100 & Wrap-around, Tie-breaker, ลำดับ `ModNHotelSystem`	ผ่าน (7/7)
`tests/test_task2_guest.py`	9	Cantor Pairing 5 กรณี, ไม่มี `last_room + 1`, ความปลอดภัยรหัสขนาดใหญ่, ค้นหาสองทาง, Atomic Batch Rollback, Edge Cases แขก	ผ่าน (9/9)
`tests/test_task3_migration.py`	10	ย้ายเฉพาะเข้าตึกใหม่เมื่อเพิ่ม / ไม่กระทบตึกอื่นเมื่อลบ ($V > 1$), เบอร์ห้องคงเดิม, ผลรวมแขกคงที่, สูตร Migration Rate, สถิติ Load Balance, เรียงห้องพัก, Edge Cases อาคาร	ผ่าน (10/10)
`tests/test_task4_integration.py`	4	จำนวนแถวและ Header ของ CSV (`guests.csv`, `migrations.csv`), ความสม่ำเสมอของตัวสร้างแขก 10 ช่องทาง, Smoke Test ของ `ExperimentRunner`	ผ่าน (4/4)

7. วิธีทดลอง ผลการทดลอง กราฟ และการอภิปรายผล

7.1 ระเบียบวิธีทดลองและโปรโตคอลการวัดผล

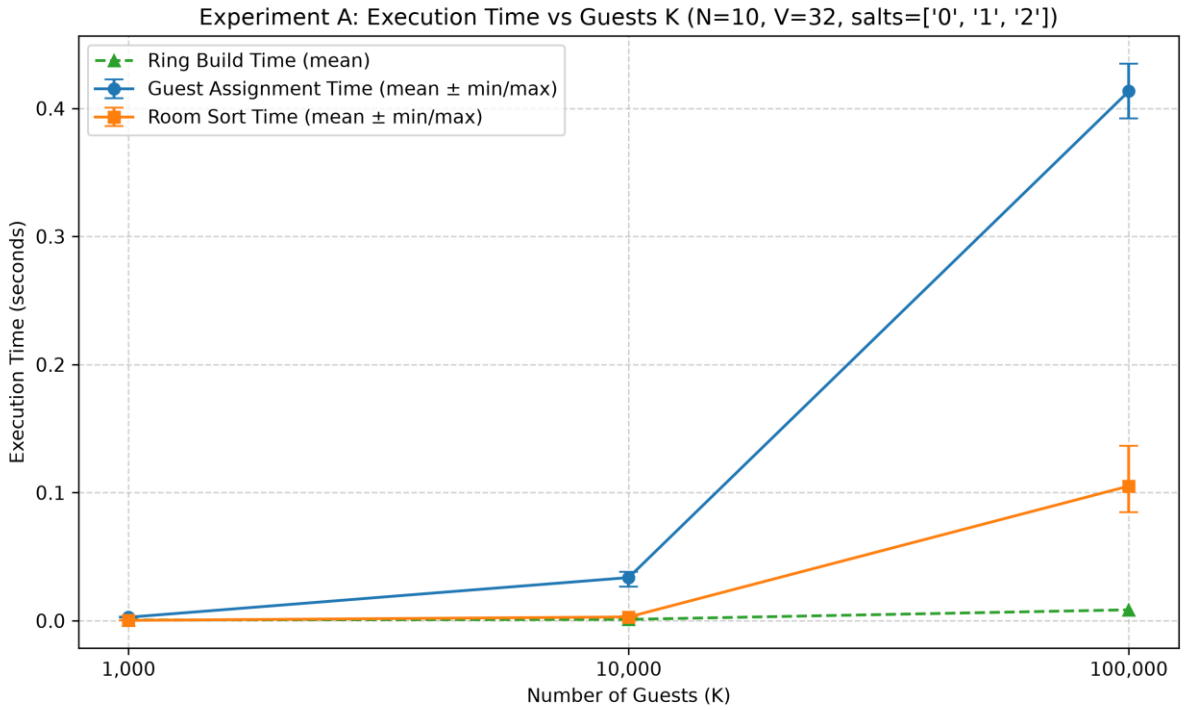
1. **ชุดข้อมูลแขกมาตรฐาน**: สร้างแขกจาก 10 ช่องทาง ($c = 1 \ldots 10$) กระจายจำนวนเท่ากันทุกช่องทางผ่าน `ExperimentRunner.generate_guest_batch(K)`

2. **การควบคุมความแปรปรวนและทำซ้ำ (Reproducibility)**: ทุกการทดลองรันซ้ำ 3 รอบด้วยค่า Salt `"0"`, `"1"`, `"2"` (ชุดเดียวกันทั้ง Consistent Hashing และ Hash Mod N ในรอบเดียวกัน) แล้วรายงานค่า **มัธยฐาน (Median)** คู่กับค่าเฉลี่ยและช่วงต่ำสุด–สูงสุด (Min–Max)
3. **การแยกวัดเวลาและหน่วยความจำ (Decoupled Measurement)**:
   - **เวลาประมวลผล**: วัดด้วย `time.perf_counter()` โดยปิด `tracemalloc` เพื่อไม่ให้เกิด Overhead จากตัวติดตามหน่วยความจำ และไม่รวมเวลา I/O หรือแสดงผล
   - **หน่วยความจำ**: วัดในรอบแยกต่างหากด้วย `tracemalloc.start()` ถึง `tracemalloc.get_traced_memory()` รายงานทั้ง `current` และ `peak` ในหน่วย MiB (ระบุชัดว่าเป็นหน่วยความจำที่ Python ติดตาม ไม่ใช่หน่วยความจำทั้งโปรเซสของระบบปฏิบัติการ)

7.2 ผลการทดลองชุด A: ผลกระทบของขนาดข้อมูล ($K \in \{1,000, 10,000, 100,000\}$, $N = 10$, $V = 32$)

ตาราง 7.1: ผลรายรอบ (Salt **0**, **1**, **2**) และค่ามัธยฐานของการทดลองชุด A (**experiment_a.csv**)

K	Salt	Ring Build (s)	Assignment (s)	Search 1,000 (s)	Search / Query (μs)	Sort Rooms (s)	Current Mem (MiB)	Peak Mem (MiB)
1,000	"0"	0.000606	0.003131	0.000099	0.099	0.000309	0.436567	0.436731
1,000	"1"	0.000571	0.002590	0.000104	0.104	0.000344	0.300261	0.300486
1,000	"2"	0.000581	0.002556	0.000097	0.097	0.000241	0.297926	0.298151
1,000	Median	0.000581	0.002590	0.000099	0.099	0.000309	0.300261	0.300486
10,000	"0"	0.000597	0.038083	0.000108	0.108	0.003390	4.125692	4.125858
10,000	"1"	0.001322	0.026797	0.000112	0.112	0.003017	4.085020	4.085186
10,000	"2"	0.001230	0.036335	0.000105	0.105	0.002529	4.084196	4.084362
10,000	Median	0.001230	0.036335	0.000108	0.108	0.003017	4.085020	4.085186
100,000	"0"	0.001362	0.413115	0.000158	0.158	0.136568	46.757307	46.861036
100,000	"1"	0.012790	0.392283	0.000174	0.174	0.084804	47.320211	47.320379
100,000	"2"	0.011504	0.434871	0.000167	0.167	0.094093	47.038931	47.039099
100,000	Median	0.011504	0.413115	0.000167	0.167	0.094093	47.038931	47.039099



รูปที่ 7.1 (Graph A): กราฟเปรียบเทียบเวลาจัดแขก เวลาเรียงลำดับห้อง และเวลาสร้างวงแหวน เมื่อเพิ่มจำนวนแขก K ($N = 10, V = 32$, แสดง Mean และช่วง Min–Max จาก 3 รอบ)

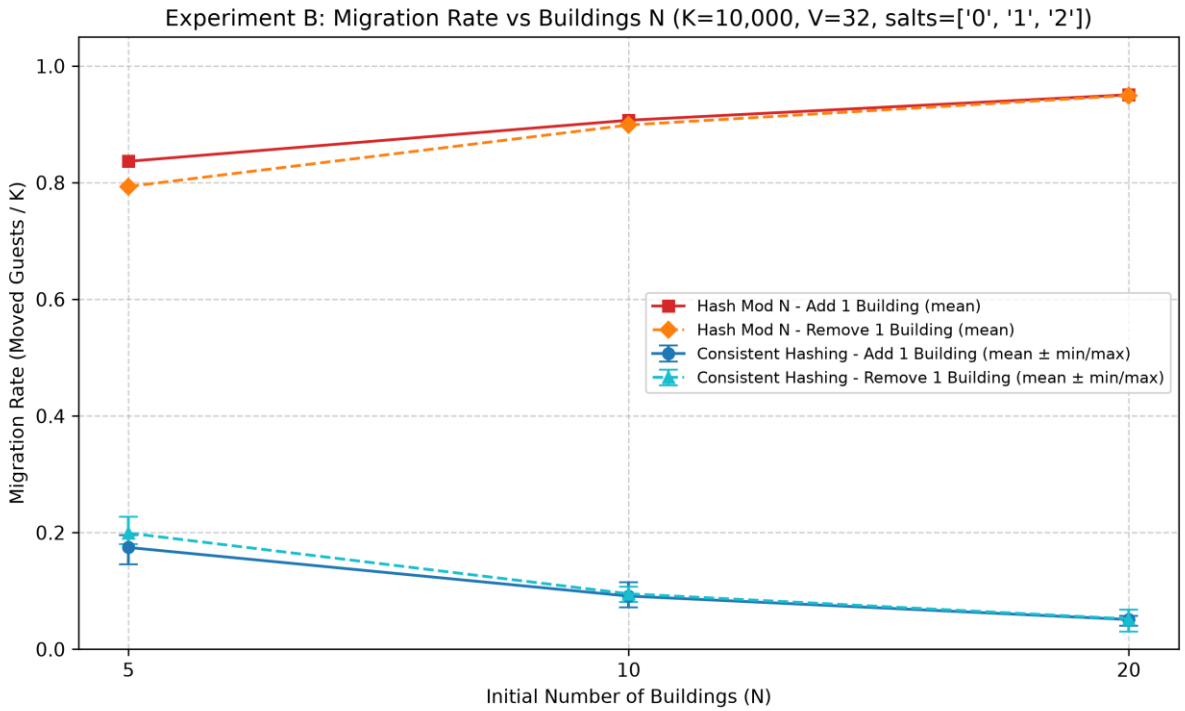
อภิปรายผลชุด A:

- เวลาจัดสรรแขก (`assignment_time`) เติบโตแบบเชิงเส้นเทียบกับ K ($0.00259\ \text{s} \rightarrow 0.03634\ \text{s} \rightarrow 0.41312\ \text{s}$) สอดคล้องกับ $\mathcal{O}(K \log M)$ โดยเมื่อ $M = 320$ คงที่ $\log_2 M$ เป็นค่าคงที่
- เวลาค้นหา $1{,}000$ ครั้งผ่าน `search_by_guest` อยู่ที่ระดับ 0.099–$0.167\ \mu s$ ต่อครั้ง ซึ่งคงที่ใกล้เคียง $\mathcal{O}(1)$ เพราะใช้ `dict` (ที่ $K = 100{,}000$ เวลาเพิ่มเล็กน้อยจาก $0.108\ \mu s$ เป็น $0.167\ \mu s$ จากผลของ L2/L3 CPU Cache Miss เมื่อตารางแฮชใหญ่ขึ้นถึง ~ 47 MiB)
- หน่วยความจำ (`peak_mem_mib`) ขยายเป็นเชิงเส้นตาม K จาก $0.30\ \text{MiB} \rightarrow 4.09\ \text{MiB} \rightarrow 47.04\ \text{MiB}$ สอดคล้องกับ $\mathcal{O}(NV + K)$

7.3 ผลการทดลองชุด B: อัตราการย้ายเทียบกับ Hash Mod N ($N \in \{5, 10, 20\}$, $K = 10,000$, $V = 32$)

ตาราง 7.2: ผลรายรอบและค่ามัธยฐานของการทดลองชุด B (**experiment_b.csv**)

N	Salt	CH Add Moved (คน)	CH Add Rate	CH Add Time (s)	CH Rem Moved (คน)	CH Rem Rate	CH Rem Time (s)	Mod N Add Moved (คน)	Mod N Add Rate	Mod N Add Time (s)	Mod N Rem Moved (คน)	Mod N Rem Rate	Mod N Rem Time (s)
5	"0"	1,456	0.1456	0.018514	2,273	0.2273	0.006240	8,314	0.8314	0.010735	7,977	0.7977	0.010702
5	"1"	1,821	0.1821	0.019636	1,803	0.1803	0.004617	8,329	0.8329	0.010844	7,936	0.7936	0.010639
5	"2"	1,960	0.1960	0.017944	1,901	0.1901	0.004919	8,457	0.8457	0.011733	7,892	0.7892	0.010875
5	Median	1,821	0.1821	0.018514	1,901	0.1901	0.004919	8,329	0.8329	0.010844	7,936	0.7936	0.010702
10	"0"	872	0.0872	0.017341	1,069	0.1069	0.003018	9,076	0.9076	0.010619	9,033	0.9033	0.010641
10	"1"	1,146	0.1146	0.017667	813	0.0813	0.002221	9,062	0.9062	0.010647	8,957	0.8957	0.010622
10	"2"	719	0.0719	0.017463	973	0.0973	0.002964	9,075	0.9075	0.010565	8,979	0.8979	0.010609
10	Median	872	0.0872	0.017463	973	0.0973	0.002964	9,075	0.9075	0.010619	8,979	0.8979	0.010622
20	"0"	405	0.0405	0.017322	684	0.0684	0.001840	9,519	0.9519	0.010185	9,488	0.9488	0.010442
20	"1"	576	0.0576	0.017618	303	0.0303	0.000899	9,487	0.9487	0.010579	9,489	0.9489	0.010133
20	"2"	549	0.0549	0.017647	576	0.0576	0.001642	9,518	0.9518	0.010936	9,510	0.9510	0.010196
20	Median	549	0.0549	0.017618	576	0.0576	0.001642	9,518	0.9518	0.010579	9,489	0.9489	0.010196



รูปที่ 7.2 (Graph B): กราฟเปรียบเทียบอัตราการย้ายแขก (Migration Rate) ระหว่าง Consistent Hashing กับ Hash Mod N เมื่อเปลี่ยนจำนวนอาคาร N ($K = 10{,}000, V = 32$)

อภิปรายผลชุด B และเหตุผลที่ผลจริงต่างจากค่าคาดหวังทางทฤษฎี:

- **แนวโน้มที่ชัดเจน**: เมื่อ N เพิ่มจาก $5 \rightarrow 10 \rightarrow 20$ อัตราการย้ายของ Consistent Hashing ลดลงอย่างมากจาก $\approx 18.2\% \rightarrow 8.7\% \rightarrow 5.5\%$ ขณะที่ Naive `hash mod N` พุ่งสูงขึ้นจาก $\approx 83.3\% \rightarrow 90.8\% \rightarrow 95.2\%$
- **เหตุใดค่าที่วัดได้จริงจึงต่างจาก $\frac{1}{N+1}$ และ $\frac{1}{N}$ เล็กน้อย**:
  - ค่าทางทฤษฎี $\frac{1}{N+1}$ (เช่น $0.1667, 0.0909, 0.0476$) และ $\frac{1}{N}$ (เช่น $0.2000, 0.1000, 0.0500$) เป็น **ค่าคาดหวังเมื่อจุดบนวงแหวนแบ่งช่วงพื้นที่ได้สม่ำเสมอสมบูรณ์และ $K \rightarrow \infty$**
  - ในการทดลองจริงใช้จำนวนโหนดเสมือนจำกัดที่ $V = 32$ ทำให้จุดเสมือนของ `Node_n` (อาคารที่เพิ่ม) หรือ `Node_0` (อาคารที่ลบ) เกิดการรวมกลุ่มเชิงสุ่ม (Random Clustering) จากผลของ SHA-256 ตามค่า `salt` ที่ต่างกัน ส่งผลให้พื้นที่รับผิดชอบรวมของอาคารนั้นกว้างหรือแคบกว่าค่าเฉลี่ยได้ เช่น ที่ $N = 10$ (เพิ่มอาคาร) ค่ามัธยฐานอยู่ที่ 0.0872 (ช่วง 0.0719–0.1146) เทียบกับค่าคาดหวัง 0.0909 และที่ $N = 20$ (ลบอาคาร) มีช่วงกว้างระหว่าง 0.0303 ถึง 0.0684 เทียบกับค่าคาดหวัง 0.0500
- **เวลาในการเพิ่มเทียบกับลบอาคาร**: เวลา `ch_add_time` (≈ 17.5–18.5 ms) คงที่แทบไม่ขึ้นกับ N เพราะใช้ Method A สแกนแขกทั้ง $K = 10{,}000$ คน แต่ `ch_rem_time` ลดลงเร็วตาม $1/N$

(4.92 ms → 2.96 ms → 1.64 ms) เพราะโค้ดวนเฉพาะแขก $\Delta K \approx K/N$ คนใน `departing_node.guests` เท่านั้น

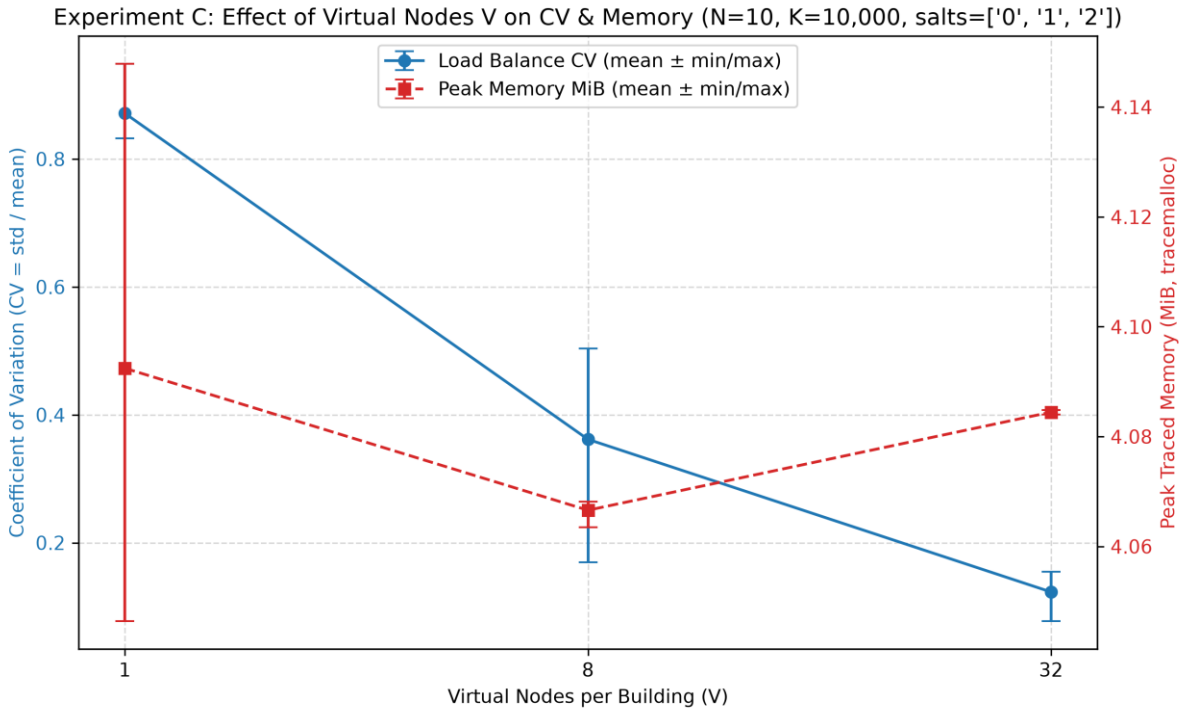
7.4 ผลการทดลองชุด C: การปรับจูน Virtual Nodes ($V \in \{1, 8, 32\}$, $N = 10, K = 10,000$)

ตาราง 7.3: ผลรายรอบและค่ามัธยฐานของการทดลองชุด C (**`experiment_c.csv`**)

V	Salt	Min Guests	Max Guests	Std Dev (σ)	$CV = \sigma/\mu$	Current Mem (MiB)	Peak Mem (MiB)	Add Moved (Rate)	Rem Moved (Rate)
1	"0"	36	2,640	833.0379	0.833038	4.147696	4.147861	377 (0.0377)	2,215 (0.2215)
1	"1"	53	3,409	948.8465	0.948846	4.082914	4.083080	748 (0.0748)	603 (0.0603)
1	"2"	104	2,512	832.5425	0.832542	4.046194	4.046360	819 (0.0819)	2,512 (0.2512)
1	Median	53	2,640	833.0379	0.833038	4.082914	4.083080	748 (0.0748)	2,215 (0.2215)
8	"0"	417	2,057	503.6264	0.503626	4.067968	4.068134	874 (0.0874)	2,057 (0.2057)
8	"1"	683	1,353	169.5600	0.169560	4.051222	4.063490	917 (0.0917)	1,029 (0.1029)
8	"2"	561	1,889	411.9903	0.411990	4.067961	4.068127	578 (0.0578)	1,526 (0.1526)
8	Median	561	1,889	411.9903	0.411990	4.067961	4.068127	874 (0.0874)	1,526 (0.1526)
32	"0"	867	1,107	77.2023	0.077202	4.084356	4.084522	872 (0.0872)	1,069 (0.1069)
32	"1"	753	1,193	154.8929	0.154893	4.083838	4.084003	1,146 (0.1146)	813 (0.0813)
32	"2"	848	1,237	137.1138	0.137114	4.084639	4.084805	719 (0.0719)	973 (0.0973)
32	Median	848	1,193	137.1138	0.137114	4.084356	4.084522	872 (0.0872)	973 (0.0973)

ตัวอย่างการกระจายจำนวนแขกต่ออาคารในรอบ `salt = "0"` ($\mu = 1{,}000.0$ คนต่ออาคาร):

- ที่ $V = 1$: `Node_5` รับโหลดสูงสุดถึง $2{,}640$ คน ขณะที่ `Node_8` มีเพียง 36 คน (ต่างกันกว่า 73 เท่า!) และเมื่อลบ `Node_0` ออกต้องย้ายถึง $2{,}215$ คน (22.15%)
- ที่ $V = 32$: อาคารทุกหลังรับโหลดสม่ำเสมอในช่วง 867–$1{,}107$ คน (`Node_0`: $1{,}069$, `Node_1`: $1{,}107$, `Node_2`: 978, `Node_3`: $1{,}093$, `Node_4`: 926, `Node_5`: 910, `Node_6`: 987, `Node_7`: 867, `Node_8`: $1{,}011$, `Node_9`: $1{,}052$)



รูปที่ 7.3 (Graph C): กราฟแสดงผลของจำนวน Virtual Nodes (V) ต่อความสมดุลของโหลด (CV) และหน่วยความจำ Peak (`tracemalloc`*) ที่ $N = 10, K = 10{,}000$*

อภิปรายผลชุด C (ข้อแลกเปลี่ยนระหว่าง CV กับหน่วยความจำ):

- การเพิ่ม V จาก $1 \rightarrow 8 \rightarrow 32$ ช่วยลดค่ามัธยฐานของ CV ลงถึง 6 เท่า จาก $0.8330 \rightarrow 0.4120 \rightarrow 0.1371$ และยังทำให้อัตราการย้ายเมื่อลบอาคารมีเสถียรภาพขึ้นมาก (แคบลงจากช่วง 0.0603–0.2512 ที่ $V = 1$ เหลือ 0.0813–0.1069 ที่ $V = 32$)
- ด้านหน่วยความจำ `tracemalloc` ในช่วง $V \in [1,32]$ มีค่า Peak ใกล้เคียงกันมากที่ ≈ 4.06–4.08 MiB (ความผันผวนระดับ ± 0.04 MiB ใน `V=1` เกิดจากการจัดสรร Table ของ `dict` ใน `HotelNode` ที่ขนาดใหญ่ไม่เท่ากันเมื่อโหลดเบ้สุดขีด) ทั้งนี้เพราะจุดเสมือน $M = 10 \times 32 = 320$ จุดใช้พื้นที่เพียงไม่กี่สิบ KB เมื่อเทียบกับออบเจกต์แขก $K = 10{,}000$ คน การตั้งค่า $V = 32$ จึงให้สมดุลที่คุ้มค่ามากในระดับ $N \leq 20$

8. ข้อจำกัดของระบบและแนวทางพัฒนาต่อ (Limitations & Future Improvements)

1. **การสแกนแขกทุกคนใน `add_building` (Method A vs Method B)**:
   - ปัจจุบัน `HotelSystem.add_building` ใช้วิธีตรวจแขกทุกคน (Method A) ทำให้เสียเวลา $\mathcal{O}(K\log M)$ แม้จะมีแขกย้ายจริงเพียง $\Delta K \approx \frac{K}{N+1}$ คน หาก K ระดับหลายล้านคนจะเกิดความหน่วงได้
   - **แนวทางพัฒนาต่อ**: จัดเก็บแขกเพิ่มเติมในโครงสร้างข้อมูลเรียงลำดับตามค่าแฮชบนวงแหวน (เช่น Self-balancing BST หรือ Sorted List/Skip List) เพื่อทำ Range Query แบบ **Method B (แบบ ข)** ดึงเฉพาะแขกที่อยู่ในช่วงวงแหวนด้านหน้าโหนดเสมือนใหม่ทั้ง V จุด ลดต้นทุนการหาผู้ย้ายลงเหลือ $\mathcal{O}(V\log K + \Delta K)$
2. **ข้อจำกัดหน่วยความจำเครื่องเดียว (Single-Machine RAM Limit)**:
   - โปรแกรมจำลองทุกอาคารและแขกทั้งหมดไว้ในหน่วยความจำหลักของเครื่องเดียว ($\mathcal{O}(NV + K)$) ที่ $K = 100{,}000$ ใช้หน่วยความจำ Python ประมาณ 47.04 MiB ดังนั้นในเครื่องที่มี RAM จำกัดจะรองรับ K ได้ระดับหลายล้านคนก่อนเกิด Memory Pressure และไม่มีระบบ Persistence/Disk-backed Storage
3. **ขอบเขตการวัดของ `tracemalloc`**:
   - `tracemalloc` วัดเฉพาะหน่วยความจำ Heap ที่ตัวแปลภาษา Python ติดตามการจัดสรรได้ ไม่รวม Overhead ของ Python Interpreter, ไลบรารีภาษา C ภายนอก หรือ Resident Set Size (RSS) ทั้งโปรเซสของระบบปฏิบัติการ
4. **ข้อเสนอแนะเชิงสถาปัตยกรรมเพิ่มเติม**:
   - รองรับ **Weighted Virtual Nodes** (กำหนดค่า V แตกต่างกันตามความจุจริงของแต่ละอาคาร) และปรับ `HashRing` ให้รองรับการสร้างวงแหวนเริ่มต้นแบบ Batch (`build_ring(node_ids)`) เพื่อเรียงลำดับครั้งเดียวใน $\mathcal{O}(M\log M)$ แทนการเรียงซ้ำ N ครั้ง

9. แหล่งอ้างอิงและการนำไปใช้จริง (References & Application)

1. Veritasium — *The Infinite Hotel Paradox*: `https://youtu.be/HLTjDXT9SqQ`
   - *การนำไปใช้*: ใช้ทำความเข้าใจข้อจำกัดของโรงแรมฮิลเบิร์ตคลาสสิกที่ต้องเลื่อนแขกทั้งระบบ ($\Theta(n)$) เมื่อมีแขกใหม่ เพื่อตั้งโจทย์สถาปัตยกรรมที่หมายเลขห้องคำนวณคงที่และแขกเดิมไม่ย้ายห้องเมื่อรับแขกใหม่
2. ByteByteGo / Hussein Nasser — *Consistent Hashing Explained*: `https://www.youtube.com/watch?v=UF9Iqmg94tk`

- *การนำไปใช้*: ใช้ออกแบบวงแหวนแฮชขนาด 64 บิต กติกาการค้นหาตามเข็มนาฬิกา (Clockwise Lookup + Wrap-around) และการใช้ Virtual Nodes (V) เพื่อลดความแปรปรวนของโหลด (CV) ในคลาส `HashRing`

3. Python Software Foundation — ***`hashlib` (SHA-256 Deterministic Hashing)***: `https://docs.python.org/3/library/hashlib.html`
   - *การนำไปใช้*: ใช้แทน `hash()` ในตัวของ Python เพื่อสร้างตำแหน่งแฮชขนาด 64 บิตแบบคงที่ข้ามการเปิดโปรแกรมใหม่ และใช้ต่อ `salt` ในการทดลองทั้ง 3 รอบ
4. Python Software Foundation — ***`bisect` (Array Bisection Algorithm)***: `https://docs.python.org/3/library/bisect.html`
   - *การนำไปใช้*: ใช้ `bisect_left` ค้นหาตำแหน่งโหนดเสมือนที่ $\geq$ guest_pos ในเวลา $\mathcal{O}(\log M)$ ร่วมกับการจัดการ Tie-breaker ด้วยทูเพิล `(position, node_id, j)` และตระหนักถึงต้นทุน Timsort/list shift เมื่อมีการปรับวงแหวน
5. Python Software Foundation — ***`time.perf_counter`***: `https://docs.python.org/3/library/time.html#time.perf_counter`
   - *การนำไปใช้*: ใช้จับเวลาความละเอียดสูงใน `ExperimentRunner` โดยแยกจับเวลาเฉพาะส่วนคำนวณล้วน (Ring Build, Assignment, Search 1,000, Sort, Add/Remove Building) ออกจาก I/O และการวัดหน่วยความจำ
6. Python Software Foundation — ***`tracemalloc`***: `https://docs.python.org/3/library/tracemalloc.html`
   - *การนำไปใช้*: ใช้บันทึกหน่วยความจำ `current` และ `peak` (MiB) ที่ Python ติดตามในรอบแยกจากการจับเวลา เพื่อให้ผลเวลาไม่ถูกรบกวนจาก Overhead ของ Profiler